codingninjas

BASIC UNDERSTANDING OF APIs

Have you ever wondered how you're able to access information from anywhere in the world—instantly? You simply type **www.google.com**, and within seconds, the search bar is ready. Or maybe you type **codingninjas.com**, and in a blink, you're on our website.

But stop and think: How does this actually happen?
What magical steps take place behind the scenes to bring a website from some server in another corner of the world, right into your screen?

It may feel like magic, but it's actually the power of the Internet Protocol Stack working in harmony

So the very first thing your browser needs to know is **where exactly this website is?** It only has the name: **www.codingninjas.com**. But the internet doesn't use names—it uses **IP addresses**.
Just like when you want to call a friend, you need their number, not just their name.

So, the first step is called **DNS Resolution**—let's see how your computer finds the IP address of the website.

1. DNS Resolution: Translating Domain Names to IP Addresses

DNS (Domain Name System) is like the **phonebook of the internet**. Humans prefer domain names (www.google.com), but machines need IP addresses (142.250.190.36) to route data.

Why is it needed?

You can't expect users to remember the IPs of every website. DNS allows us to use easy-to-remember domain names.

How it Works (Step-by-Step):

1. You type www.example.com into your browser.
2. The browser checks the **local DNS cache** (if you've visited recently).
3. If not found, it queries the **recursive resolver** (usually provided by your ISP).
4. Resolver asks the **root DNS server** → then the **TLD server** (like .com) → then the **authoritative DNS server**.
5. The final IP address is returned to your computer.
6. Now, your browser knows where to send the request.

For Example:

codingninjas

Imagine you want to call **Pizza Hut** but don't know the number. You check your contact list (cache), then try Google (recursive resolver), which checks various directories until it gets the number.

Now that your computer **knows the IP address**, it's like having the phone number of a friend.
But before you start talking (or exchanging data), you need to establish a connection, just like how we say 'hello' and wait for the other person to respond on a call.

This brings us to our second step: the **TCP Handshake**.

2. TCP Handshake: Establishing a Reliable Connection

TCP (Transmission Control Protocol) ensures that **data is sent reliably and in order**. Before any actual data transfer, a connection must be established using the **three-way handshake**.

The 3-Step Handshake:

1. **SYN (Synchronize)** – Client says, "Hey, I want to talk. Here's my sequence number."
2. **SYN-ACK** – Server replies, "Sure! Here's my sequence number. Acknowledging yours too."
3. **ACK (Acknowledge)** – Client says, "Cool, ready to start!"

Only after this process can data transfer begin.

For Example:

Think of it like a polite phone call:

- You dial a number (SYN)
- They pick up and say, "Hello?" (SYN-ACK)
- You say, "Hi, can we talk now?" (ACK)
- Now the conversation starts.

Great! So now both your browser and the server are ready to talk. The connection has been established.
Now comes the actual conversation—**the data transfer**.

Think of it like you're now ready to send a large letter, but it won't fit in one envelope.
So what do we do? We split it into parts and send them one by one.

That's exactly what happens in Step 3: **Breaking and Sending Packets**.

3. Data Transfer: Breaking and Sending Packets

So, once the connection is established, actual **data (like an HTML page, Files, Video, Audio)** is transferred in small chunks called **packets**.

What is a Packet?

A **packet** is a small unit of data that travels over the Internet.

Whenever you send or receive something online — a message, a web page, a video — that data is **broken into small pieces** called **packets** before being sent.

Why Break Data into Packets?

Think of it like this:

> You want to send a large book by post, but the courier only accepts small envelopes.
> So, you tear out each page, number them, and send them in separate envelopes.

That's exactly what the internet does — it breaks large data into small, numbered packets and sends them individually.

What Does a Packet Contain?

Each packet carries two main parts:

1. Header (Label/Instructions)

Like the envelope that wraps your page.

This includes:

- **Source IP address**: Who sent the packet
- **Destination IP address**: Where the packet is going
- **Packet number**: So the receiver knows the order
- **Protocol info**: How the data should be handled (e.g, TCP or UDP)
- **Error-checking code**: To check if the packet got damaged

Note:

What is an IP Address?

An **IP address** is like the **home address of your device** on the internet.

It tells the internet **where to send data**, just like your home address tells a delivery guy where to send your pizza.

But…

Have you heard about another kind of address called a MAC address?

It's also a type of address, but it works very differently from an IP.

What is a MAC Address?

While your IP address is like a temporary delivery address (which can change when you switch Wi-Fi or networks),
Your MAC address is more like your device's permanent identity card.

MAC stands for Media Access Control.

It is a permanent, unique identifier assigned to a device's network interface card (NIC) — the hardware that connects it to a network.

You can think of it as the fingerprint of your device's network hardware — it never changes and is always tied to that physical device

So now the second thing the packet contains is:

2. Payload (Actual Data)

This is the **real content**, like:

- Part of your email
- A section of a YouTube video
- A chunk of a webpage
  It's the *useful* part the receiver needs.

Key Concepts:

- TCP divides large data into packets (usually around 1500 bytes).
- Each packet has a **header** (metadata: sequence number, source/destination) and **payload** (data).
- TCP ensures all packets arrive and are in the correct order.
- **Acknowledgments (ACKs)** are sent to confirm receipt.

For Example:

Imagine sending a long handwritten letter by post, but your envelope can only carry 1 page at a time. You number each page (sequence number), and your friend confirms receipt of each one.

Example in a Web Scenario:

You request a 500KB image from a server.
It's broken into ~340 packets (assuming ~1500B per packet).
Packets are sent individually and might take different routes to reach you.

So now, your browser starts receiving small data packets, just like getting envelopes with pages of a letter.

But remember: packets may come **out of order**, or **one might get lost**.

So, how does your computer make sense of this messy pile?

Welcome to Step 4: **Reassembly and Processing**—where everything is put back in the correct order before being shown to you.

4. Reassembly and Processing: Putting the Packets Back Together

What Happens at the Destination?

- The receiver (browser or app) **reassembles** all the packets using the sequence numbers.
- If a packet is missing or corrupt, it **requests a retransmission**.
- Once complete, the browser renders the content (e.g., an image, webpage, etc.).

For Example:

You receive all pages of the letter and arrange them in the right order before reading. If page 5 is missing, you call your friend to resend it.

So you've got the complete webpage now. You're seeing the images, reading the text—it's all done.

Now that the job is finished, it's time to say goodbye.

Just like ending a phone call, we don't just hang up immediately. There's a proper way to close the connection to avoid any leftover noise.

That's what happens in Step 5: **Connection Termination**."

5. Connection Termination: Closing the Communication

Why Close the Connection?

Once the data transfer is done, both sides agree to **close the TCP connection** to free up resources.

The 4-Step FIN-ACK Process:

1. Client sends **FIN** – "I'm done sending data."
2. Server replies with **ACK** – "Got it."
3. Server sends its own **FIN** – "I'm also done."
4. Client replies with **ACK** – "Okay, all done!"

The connection is now closed cleanly.

For Example:

You and your friend say goodbye over a phone call:

- "Okay, I'm done."
- "Got it. I'm done too."
- "Cool. Bye!"

Notes:

- Connections not properly closed may stay in the **TIME_WAIT** state.
- TCP ensures **graceful disconnection**, preventing data loss.

Next time you open a website, take a moment to appreciate the **invisible choreography** of technology—DNS lookups, handshakes, data packets, and graceful shutdowns—**all working together** just to show you a webpage.
That's the magic of the internet—and now, you know how it works behind the scenes.

So far, we've seen the path your request takes across the internet—from DNS resolution to packet delivery.
But there's one core concept that underpins all of this—a model that determines who requests data and who responds.

Let's start with a simple but powerful concept that underlies how most modern technology works — whether you're browsing a website, sending a message, or building with AI —
And that is the **Client-Server Architecture**.

Client-Server Architecture:

Think of ordering food in a restaurant:

- You (the client) place an order
- The kitchen (the server) prepares the food
- The waiter brings your meal back to you

You don't cook the food — you just ask, and the server handles the rest.

This is how exactly a client-server architecture works, where

- The **client** makes a request
- The **server** processes the request and sends back a response

They communicate over a network, usually the internet.

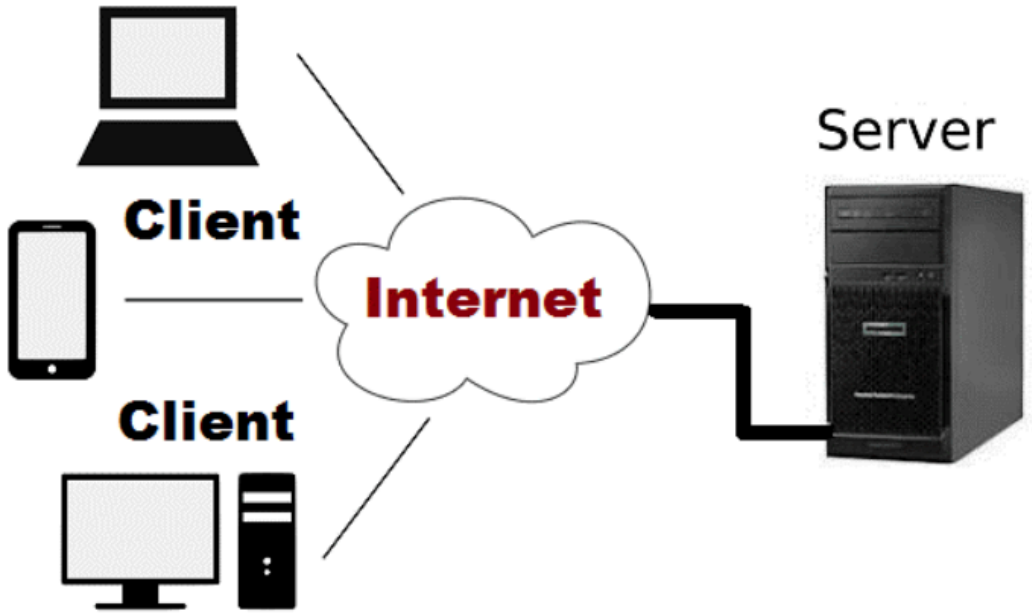


Figure 1: Client-Server Framework

Now that we understand how the client-server architecture works — where the client sends a request and the server sends back a response — the next question is:

> *How do the client and server communicate in a structured way?*

That's where an API comes in.

What is an API?

API stands for **Application Programming Interface**.
It's a set of rules or a communication bridge that allows one software or system to talk to another.

You can think of it as a **messenger** that:

- Takes your request to the system
- Brings back the response in a way your app can understand

Let's understand it with the basic example that we have seen before.

Imagine you're sitting in a restaurant.

- You don't walk into the kitchen or prepare your own food.
- Instead, you look at the menu and tell the waiter what you'd like to eat.
- The waiter takes your order to the kitchen, where the food is prepared.
- Once it's ready, the waiter brings the food back to your table.

Now here's how this maps to a technical system:

- You are the client (your app or code)
- The waiter is the API — your point of communication
- The kitchen is the server (e.g., Gemini or ChatGPT model)
- The food is the response you receive

You didn't need to know how the kitchen works — you just made a request, and it returned a result.

This is how an API works

The API doesn't do the actual work (like cooking), but it understands your request, delivers it to the server, and brings back the result in a way you can understand.

Just like you trust the waiter to talk to the kitchen, your app uses an API to talk to the model server — you don't need to know how the server works internally

We've understood that an **API is like a messenger** — it takes your request from a client (like your browser or app), goes to the server (like Gemini, ChatGPT), and brings the result back to you.

But wait a second…

Where does this server actually exist?

How does it respond to millions of LLM queries every single day?
How does it *scale* to serve thousands of users at the same time?

Let's now look **behind the scenes** — into the **modern architecture** that powers these APIs.
The server (which is the brain behind the API) can be hosted in two major ways:

Serverful APIs — Traditional, always-on infrastructure
Serverless APIs — Modern, event-driven, cloud-native infrastructure

Each plays a crucial role depending on the **nature of the task and system**, and both are used to serve LLMs, sometimes in combination.

Let's break them down.

1. What is a Serverful API?

A **Serverful API** is hosted on a **dedicated server or virtual machine (VM)**.
This server is **always running**, whether or not any request is being made.

Think of it like a restaurant kitchen that's open 24x7 — even when no one is ordering.

In this model:

- You (or the provider) must **set up the infrastructure**: the machine, memory, CPU, scaling, etc.
- The server is **always available** to take requests, like an always-open shop.
- All LLM computation (e.g., generating embeddings or responses) happens on this machine.

LLM Example: Using a Local Model via API

Imagine you download a model like **LLaMA or Mistral**, deploy it on your own VM(Virtual Machine), and expose it via a Flask or FastAPI server.
This is a classic serverful setup. You must:

- Keep the server running,
- Manage memory usage for large models,
- Monitor uptime and failures.

Advantages of Serverful APIs:

- **Full control** over hardware, performance, and model configuration.
- **Consistent latency** because the server is always warm.
- Good for **LLMs fine-tuned for specific tasks** running on enterprise hardware.

Disadvantages:

- High cost: You pay even when no one uses the API.
- Poor scalability: Handling spikes in LLM usage requires manual scaling or expensive autoscaling setups.
- Higher responsibility: You manage security patches, infra failures, and updates.

But what if you **don't want to manage any servers**?
What if you want your code to just **run automatically when a request comes**, and not waste money or effort on idle machines?

That's where **Serverless APIs** come in — the modern backbone behind most **LLM-as-a-service platforms**.

2. What is a Serverless API?

A **Serverless API** means:

> You write your logic, deploy it to the cloud, and the cloud handles everything else.

There's *still a server* behind the scenes, but you **never manage it**.

The server **spins up on demand**, executes your code (like processing a prompt), then shuts down.
You **only pay for the time used**, not for idle waiting.

LLM Example: Calling Gemini or ChatGPT APIs

When you call Gemini LLM using the GEMINI SDK,

you're using a **serverless endpoint**.
Google's infrastructure automatically:

- Spins up LLM inference engines,
- Loads the model weights,
- Processes your prompt,
- Sends back the output,
- And tears down the computer — all within milliseconds.

You didn't manage the server — you just called the API.

Advantages of Serverless APIs:

- **Zero infrastructure management** — just write code.
- **Cost-effective** — pay only for usage.
- **Elastic scalability** — handles 1 or 1 million LLM calls without you doing anything.
- Ideal for event-driven systems (e.g., AI chatbots, summarization tools, etc.)

Disadvantages:

- **Cold starts** — slight delay if the function hasn't been invoked recently.
- Less control over memory, GPU placement, or warm-up optimization.
- You rely on the provider's platform (vendor lock-in).

So, we now understand that the **API response we receive can come from two types of backend infrastructures — Serverful or Serverless**.
Each has its own strengths and trade-offs, and the choice between them entirely depends on your **use case**, performance needs, cost considerations, and how you plan to **build and manage your product**.

With that foundation in place, let's now move one step deeper.

Before we can send or receive anything through an API, we need to understand the language APIs speak — and that language is JSON.

Whether you're sending a prompt to an LLM or receiving its answer, you're dealing with **structured data in JSON format**.

What is JSON?

JSON stands for: "JavaScript Object Notation"

Despite having "JavaScript" in the name, **JSON is not a programming language** — it's a lightweight, text-based format used to **store and exchange data between systems**, especially over the internet.

Imagine filling out a form at a doctor's clinic.

- You write your **name**, **age**, **symptoms**, and **preferred appointment time**.
- The form collects all your details in a **structured format**, so the clinic staff can easily read and process your request.

Similarly, **JSON** is like a digital form that organizes data into **key-value pairs** so systems (like APIs) can understand what you're sending and return meaningful information.

For Example:

```
{
 "name": "Nikhil",
 "age": 22,
 "interests": ["AI", "Coding", "Space"],
 "is_student": true
}
```

Keys: "name", "age", "interests"
Values: "Nikhil", 22, a list of strings, and true

Now that you've understood what JSON is, i.e, a structured way to represent data using key-value pairs —

Let's actually see JSON in action by calling a real API.

Visualizing a Real API Response

We're now going to see a live example of how an **API** works and what kind of **JSON data** it returns — using nothing but your **web browser**.

codingninjas

Step-by-Step Instructions:

Step 1: Open your browser

Open Google Chrome, Firefox, or any browser of your choice.

Step 2: Paste this URL into the address bar

https://jsonplaceholder.typicode.com/users/1
Then press **Enter**.

Step 3: Observe what you see

You'll see a structured response like this:

json


```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874"
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```


What's Happening Behind the Scenes?

Let's break this down using the concepts we've learned:

Who is the Client here?

You — or more specifically, **your browser**.
When you typed in the URL and hit Enter, your browser acted as the **client**, making a request.

codingninjas

Who is the Server?

The server behind ***jsonplaceholder.typicode.com*** is a mock API service.
It receives your request for data and sends back a **JSON response** containing user information.

What is the API here?

The **API** is the URL endpoint you used:
https://jsonplaceholder.typicode.com/users/1

It defines a rule:

> "If a client asks for **/users/1**, I'll return the details of user ID 1 in JSON format."

What is JSON here?

The response you saw is a **JSON object**.
It contains data like:

- name: "Leanne Graham"
- email: "Sincere@april.biz"
- Nested fields like address, city, or company.name

So far, we've understood that an **API helps the client and server communicate** with each other.

But just like every meaningful conversation follows some rules —
API communication also follows a **standard set of rules** to ensure the client and server understand each other clearly.

This set of rules is called the **HTTP protocol**, which stands for **Hypertext Transfer Protocol**.

Why Do We Need HTTP?

HTTP acts like a **common language** or **delivery protocol** that ensures:

- The **client's request is structured properly**
- The **server knows how to read and respond**
- The communication is **fast, consistent, and reliable**

> *Think of HTTP as the postal system's guidelines — it tells us how to format, label, and send our "data parcels" from sender (client) to receiver (server).*

Now that we understand **HTTP** is the protocol that makes communication between the client and server possible, the next thing to know is **how the client tells the server what kind of action it wants to perform**.

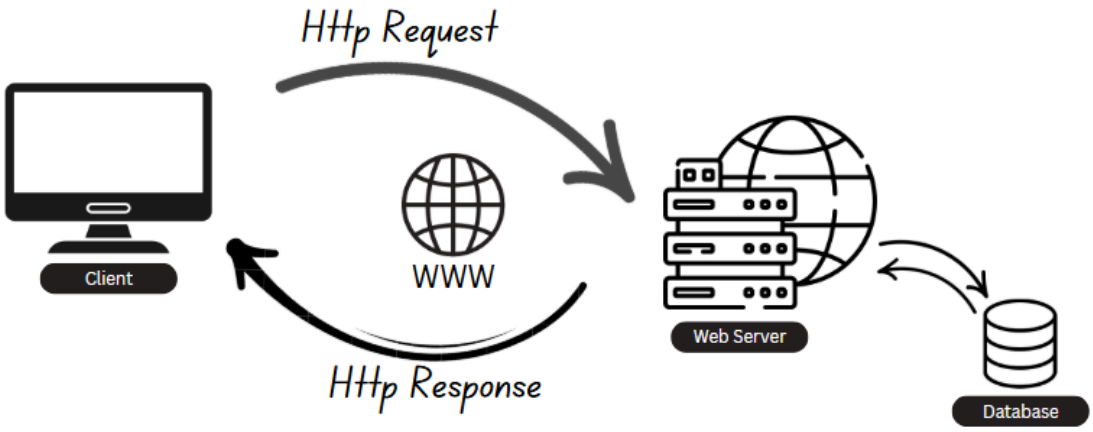
Just like in real life, we use verbs like "get", "send", or "delete" to describe actions —
In the world of APIs, we use something called **HTTP Methods**.

What are HTTP Methods?

When a **client** (like your browser or Python script) sends a request to a **server** using an API, it must also specify **what kind of action** it wants to perform.

This action is defined by the **HTTP Method**.

You can think of HTTP methods as **verbs** — they describe **what** you're trying to do with the data on the server.



Most Common HTTP Methods:

1. GET → "Give Me Something"

- Used to **retrieve** data from the server.
- You're not sending any data — just asking for information.

Example:
Getting a user's profile, list of posts, or public content

2. POST → "I'm Sending You Something to Process"

- Used to **send data** to the server (e.g., submitting a prompt to an LLM).
- The server usually processes it and returns a result or creates something new.

> *"Most **LLM APIs** (like Gemini, ChatGPT) use **POST** because you are **sending a prompt** and getting back a **generated response**."*

3. PUT → "Update Something That Already Exists"

- Used to **update** a resource completely.
- You send the full updated data.

4. PATCH → "Make a Small Change"

- Similar to **PUT**, but used to **update only part** of a resource.

5. DELETE → "Remove This"

- Used to **delete** a resource from the server.

Now that we know **how to send different types of requests** to a server using HTTP methods like **GET**, **POST,** and **DELETE** —
The next question is: **How do we know what happened to our request?**

Did it work? Did it fail? Did we make a mistake? Or did something go wrong on the server?

That's where **HTTP Status Codes** come in.

What is an HTTP Status Code?

Whenever a client (like your browser or Python script) sends a request to a server, the server responds with more than just the data —
It also sends a **three-digit status code** that tells you what happened to your request.

These codes help you understand:

- Whether the request was successful
- If there was an error
- Or if some other action is needed

Types of HTTP Status Codes:

1xx – Informational

These codes indicate that the request has been received and is still being processed.

- **100 Continue** → The server has received the request headers and is waiting for the body.

2xx – Success

These codes mean the request was successfully received, understood, and processed.

- **200 OK** → The request was successful, and the server is returning the requested data.
- **201 Created** → A new resource was successfully created (used in POST requests).

3xx – Redirection

These codes indicate that further action is needed to complete the request.

- **301 Moved Permanently** → The requested page or resource has been moved to a new URL.
- **302 Found** → Temporarily redirected to another resource.

4xx – Client Errors

These codes mean there was something wrong with the **client's request** (e.g., bad input, unauthorized access).

- **400 Bad Request** → The request was malformed or missing required data.
- **401 Unauthorized** → You need to provide a valid API key or credentials.
- **403 Forbidden** → You don't have permission to access this resource.
- **404 Not Found** → The requested resource doesn't exist.

5xx – Server Errors

These codes indicate that something went wrong on the **server's side**, not yours.

- **500 Internal Server Error** → The server had an unexpected issue.
- **503 Service Unavailable** → The server is currently overloaded or under maintenance.

The content above gives you the **essential foundation** you need to understand:

- What APIs are
- How clients and servers communicate
- What role do HTTP and JSON play in that communication
- And how status codes and methods help us manage requests and responses

This basic knowledge will help you feel more confident as we move into the **live session**, where we'll go hands-on and see **how to actually implement and interact with an LLM API** in real-time.

See you there, and get ready to build with AI!